National Taipei University of Technology

Object-Oriented Analysis and Design

Homework #2

arch-checker

A Fluent Architectural Style Compliance Tool for Java

Team Member:

113598009 資工碩二 李俊威

2026-03-12

2. Requirement Document

2.1 Change History

Version	Date	Author	Description
1.0	2026-03-03	李俊威	Initial submission (HW1) — Problem Statement
2.0	2026-03-12	李俊威	Add System Features, Use Cases, NFR, Glossary (HW2)

2.2 Problem Statement

隨著軟體系統規模不斷擴大，維護一致的架構風格成為開發團隊面臨的核心挑戰之一。架構風格是一組規範類別之間職責分配、依賴方向與互動方式的結構性約束，開發團隊通常會在專案初期選定特定的架構風格，作為整個程式碼庫的設計準則。然而，隨著程式碼持續增長，確認實際產出的程式碼是否真正符合預期的架構風格，往往只能依賴人工逐一審查，這是一個費時、主觀且難以規模化的過程。

這個問題在導入 AI 輔助程式碼生成的開發情境中尤為突出。當開發者使用大型語言模型生成程式碼時，產出物在功能上可能正確，但在結構上卻可能偏離預期的架構風格。若缺乏自動化的合規驗證工具，開發者只能在事後手動比對每個類別是否符合架構規範，大量的人工驗證時間在開發週期上變成了 **Bottleneck**。

現有工具（如 ArchUnit）雖然允許開發者將架構限制寫成測試規則，但使用門檻較高——開發者必須熟悉其底層 API，並為每一條規則撰寫繁瑣的樣板程式碼。目前市場上缺乏一種高階、以風格為中心的抽象工具，讓開發者能夠以簡潔的宣告方式定義「這個專案遵循某種架構風格」，並立即獲得具體、可操作的違規回饋。

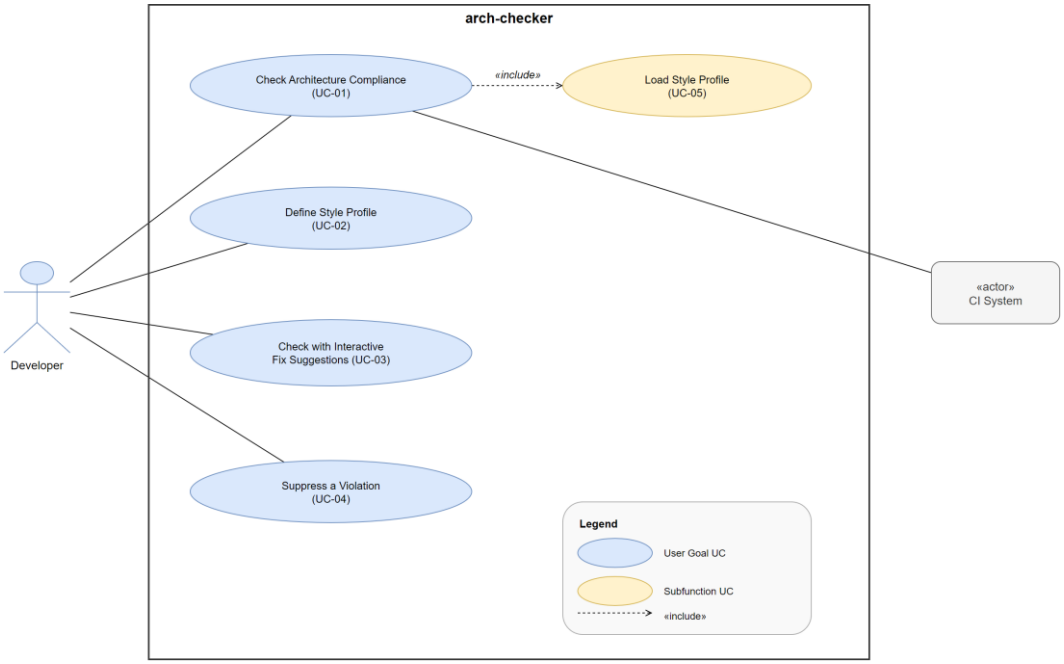
arch-checker 是一套專為 Java 開發者、軟體架構師與研究人員設計的命令列工具，用於自動驗證 Java 專案的架構風格合規性。使用者可透過流暢的宣告式 API 定義架構風格設定檔（Style Profile），將一組命名規則封裝為可重複使用的風格單元。工具內建常見架構風格的規則集，同時支援使用者自行定義任意風格的規則。系統底層透過 Java 原始碼的抽象語法樹（AST）解析，精確偵測結構性違規，並產出包含檔案路徑、行號與錯誤描述的完整報告。

arch-checker 適用於本地開發、程式碼審查與持續整合三種情境，協助開發團隊在架構偏移發生的早期即時發現問題，降低後期重構的成本與風險。

2.3 Summary of System Features

ID	Feature Name	Priority	Description
SF-01	Architecture Compliance Checking	High	掃描 Java 專案目錄，依使用者定義的 Style Profile 偵測違規
SF-02	Style Profile Definition via YAML	High	使用者以 YAML 格式定義任意架構風格規則（DependencyRule、NamingRule、SupertypeRule、PackageRule）；工具不預設任何內建風格
SF-03	Violation Report Generation	High	輸出含檔案路徑、行號、規則 ID、說明的違規報告
SF-04	Multiple Report Formats	Low	支援 Console 與 JSON 兩種輸出格式
SF-05	Violation Suppression	Medium	標記特定違規為「已知且接受」，下次不再回報

2.4 Use Case Diagram



Actors:

Developer (Primary)：使用工具定義架構規則並檢查 Java 專案的開發者

CI System (Secondary)：在 CI/CD pipeline 中自動觸發合規檢查，依賴 exit code 判斷結果

UC	Name	Level	Format	Triggered by
UC-01	Check Architecture Compliance	User Goal	Fully Dressed	Developer / CI System
UC-02	Define Style Profile	User Goal	Fully Dressed	Developer
UC-03	Check with Interactive Fix Suggestions	User Goal	Fully Dressed	Developer
UC-04	Suppress a Violation	User Goal	Brief	Developer
UC-05	Load Style Profile	Subfunction	Brief	UC-01 / UC-03 (include)

2.5 Use Cases

UC-01: Check Architecture Compliance (Fully Dressed)

Use Case	UC-01
Use Case Name	Check Architecture Compliance
Scope	arch-checker CLI
Level	User Goal
Primary Actor	Developer
Stakeholders and Interests	Developer：想要取得準確且詳細的違規報告，包含檔案路徑與行號，以便快速找出不符合規範的類別，避免費時的人工審查。 Team Lead / Architect：想確保整個程式碼庫符合團隊定義的架構規則，零違規代表架構一致性獲得維護。 CI System（次要）：想要可靠的 POSIX exit code（0/1/2）來判斷 pipeline 是否通過，不需要解讀報告內文。
Preconditions	1. arch-checker 已安裝並可在系統 PATH 中存取 2. 目標 Java 專案目錄存在且可讀取 3. <yaml-path> 指定的 Style Profile YAML 檔案存在且語法正確
Success Guarantee	系統產生完整的 ViolationReport，包含：已檢查檔案數、違規清單、suppressedCount；並回傳 POSIX exit code（0 = pass，1 = violations found）
Main Success Scenario	1. Developer 執行 arch-checker check --profile <yaml-path> <project-path> 2. 系統確認 project-path 目錄存在

	3. 系統讀取並解析 <yaml-path> 指定的 Style Profile YAML 檔案 4. 系統遞迴掃描 project-path 下所有 .java 檔案 5. 系統以 JavaParser 將每個 .java 檔解析為 AST（CompilationUnit） 6. 系統對每個 CompilationUnit 套用 Profile 中所有 ArchitectureConstraint 規則 7. 系統收集所有 Violation，每筆包含：檔案路徑、行號、規則 ID、說明文字 8. 系統過濾已被 suppress 的 Violation 9. 系統以 Console 格式輸出 ViolationReport（含違規清單與 summary） 10. 系統回傳 exit code：無違規則 0，有違規則 1
Extensions	2a. project-path 不存在：系統輸出 Error: path not found <path>，exit code 2，終止 3a. <yaml-path> 不存在：系統輸出 Error: profile file not found <path>，exit code 2，終止 3b. YAML 語法錯誤：系統輸出 YAML 解析錯誤與行號，exit code 2，終止 3c. YAML 語法正確但規則定義無效（如未知的 rule type）：系統輸出 validation 錯誤清單，exit code 2，終止 4a. project-path 下無 .java 檔：系統輸出 Warning: No Java files found in <path>，exit code 0，終止 5a. 某 .java 檔語法錯誤無法解析：系統記錄 Warning: skipped <file> (parse error)，跳過，繼續處理其餘檔案 8a. 所有 Violation 皆已被 suppress：報告顯示 All checks passed (N violations suppressed)，exit code 0
Special Requirements	NFR-01（Performance）：掃描 1,000 個 .java 檔案必須在 10 秒內完成（一般開發機） NFR-02（Usability）：所有錯誤訊息必須包含：錯誤原因、位置（檔案/行號）、建議修正動作 NFR-05（CI Compatibility）：exit code 遵循 POSIX 標準，可直接整合 GitHub Actions / Jenkins
Technology and Data Variations List	*a. --profile 接受任意 YAML 檔案路徑，不支援內建名稱 *b. 輸出格式可透過 --format json 切換為 JSON，預設為 Console 文字格式 4a. Java 檔案掃描採遞迴深度優先走訪
Frequency of Occurrence	CI 環境：每次 push 觸發，每日可達數十次；本地開發：每小時數次

Miscellaneous	是否需支援 multi-module Maven 專案（多個 src/main/java 子目錄）？ 大型專案（>10,000 個 .java 檔）是否需要增量掃描機制？

UC-02: Define Style Profile (Fully Dressed)

Use Case	UC-02
Use Case Name	Define Style Profile
Scope	arch-checker CLI
Level	User Goal
Primary Actor	Developer
Stakeholders and Interests	Developer：想用最少工夫定義出可重複使用的架構規則，希望工具提供模板降低手寫 YAML 的門檻，並在儲存前即時得知格式是否正確。 Team Lead：想要團隊共享一份統一的 Profile 並版本控制於 Git，確保規則描述清楚且長期可維護。
Preconditions	1. arch-checker 已安裝並可在系統 PATH 中存取 2. Developer 了解目標專案的架構規範（知道哪些 package 屬於哪個層、哪些依賴是被禁止的）
Success Guarantee	一個語法正確、至少含 1 條規則的 .yaml Profile 檔案存在於磁碟，且可被 UC-01 成功載入使用
Main Success Scenario	1. Developer 執行 arch-checker init-profile --output <file.yaml> 2. 系統產生含預設結構與欄位說明註解的空白 YAML 模板，寫入 <file.yaml> 3. Developer 以文字編輯器開啟 <file.yaml>，填入規則（rule type、package pattern、constraint 細節） 4. Developer 執行 arch-checker validate-profile <file.yaml> 驗證語法與規則定義 5. 系統讀取並解析 YAML 檔 6. 系統驗證每條規則的 type（NamingRule / DependencyRule / SupertypeRule / PackageRule）與所有必填欄位 7. 系統驗證每個 package pattern 的語法（只允許英數字、*、.） 8. 系統輸出 Profile valid: <N> rules loaded 9. Developer 在執行 UC-01 時以 --profile <file.yaml> 指向此 Profile
Extensions	4a. Developer 跳過 init-profile，直接對已存在的 YAML 執行 validate-profile：從 Step 5 開始，正常執行

	5a. <file.yaml> 不存在：系統輸出 Error: file not found <path>，終止 6a. YAML 頂層結構錯誤（不是合法的 YAML mapping）：系統輸出 Invalid YAML structure 與錯誤行號，終止 6b. 遇到未知的 rule type：系統輸出 Unknown rule type: <type>. Allowed: NamingRule, DependencyRule, SupertypeRule, PackageRule，終止 7a. package pattern 含非法字元：系統輸出 Invalid package pattern: <pattern>，終止 8a. YAML 格式正確但 rules 清單為空：系統輸出 Warning: Profile has no rules. Profile is valid but will produce no violations.，exit code 0
Special Requirements	NFR-02（Usability）：所有 validation 錯誤訊息必須清楚指出 YAML 錯誤行號，並提供修正建議 NFR-04（Extensibility）：新增 rule type 不需修改 ProfileValidator 核心邏輯（Open/Closed Principle）
Technology and Data Variations List	*a. Profile 格式為 YAML（以 SnakeYAML 解析），採 UTF-8 編碼 *b. Developer 可使用任意文字編輯器（VS Code、IntelliJ、Vim 等），不需 IDE plugin 2a. 若 --output 指定的目錄不存在，系統自動建立
Frequency of Occurrence	每個新專案通常執行一次；規則有異動時重複執行 validate；相對低頻，以週或月為單位
Miscellaneous	是否提供 JSON Schema 供 IntelliJ / VS Code 在編輯 Profile YAML 時自動補全？ Profile 是否支援繼承（extends: <other-yaml-path>）以在既有規則上擴充？

UC-03: Check Architecture Compliance with Interactive Fix Suggestions (Fully Dressed)

Use Case	UC-03
Use Case Name	Check Architecture Compliance with Interactive Fix Suggestions
Scope	arch-checker CLI
Level	User Goal
Primary Actor	Developer
Stakeholders and Interests	Developer：想逐條審視每個 Violation 並當場決定是否 suppress，希望系統提供可操作的修復建議，而不只是列出錯

	誤清單。 Team Lead：想知道哪些 Violation 是「已知且接受」的技術債，並有持久化且可稽核的 suppression 紀錄，而非無聲地被略過。
Preconditions	1. 同 UC-01 的前置條件 2. Developer 有意願逐條決策每個 Violation（非 CI 批次自動化情境） 3. Terminal 支援互動式輸入（非 pipe 重定向的 non-TTY 模式）
Success Guarantee	對每條 Violation，三種結果之一成立：(a) Suppression 記錄寫入 .arch-checker-suppress.yaml，(b) Violation 保留在報告中，(c) 流程中止；最終 ComplianceReport 反映所有決策結果
Main Success Scenario	1. Developer 執行 arch-checker check --interactive --profile <yaml-path> <project-path> 2. 系統完成掃描流程（同 UC-01 Step 2–7）：走訪檔案、解析 AST、套用規則、收集所有 Violation 3. 系統展示第一條 Violation，顯示：檔案路徑與行號、違反的 ArchitectureConstraint ID 與說明、RefactoringSuggestion（若存在） 4. 系統提示 Accept suppression? [y/n/q]: 5. Developer 輸入 y：系統將此 Violation 的 constraint ID 與時間戳記寫入 .arch-checker-suppress.yaml 6. 系統展示下一條 Violation（回到 Step 3） 7. 所有 Violation 處理完畢後，系統輸出最終 ComplianceReport（已扣除接受的 suppression） 8. 系統回傳 exit code（0 = 無未接受 Violation，1 = 有未接受 Violation）
Extensions	3a. 此 Violation 有 RefactoringSuggestion：系統在顯示 [y/n/q] 提示前，額外顯示建議操作 3b. 掃描後無 Violation：系統直接輸出 All checks passed，不進入互動迴圈，exit code 0 5a. Developer 輸入 n：此 Violation 保留在最終報告中，系統繼續下一條（Step 6） 5b. Developer 輸入 q：系統中止互動迴圈，輸出已處理部分的 partial report，exit code 1 5c. Developer 輸入非法值（非 y/n/q）：系統輸出 Invalid input. Please enter y, n, or q:，重新等待輸入 8a. 所有 Violation 均被 suppress（全部輸入 y）：報告顯示 All checks passed (N violations suppressed)，exit code 0

Special Requirements	NFR-02（Usability）：互動提示須清晰易懂，Developer 無需查閱文件即能理解 [y/n/q] 各選項的含義 NFR-06（Testability）：互動流程必須可在不啟動完整 CLI 的情況下以 unit test 驗證（I/O 需可 mock）
Technology and Data Variations List	*a. Suppression 記錄寫入 .arch-checker-suppress.yaml，包含：constraint ID、reason（預設為 suppressed interactively）、timestamp 4a. [y/n/q] 輸入大小寫不敏感（Y、Yes 均視為接受） *b. 若 stdout 被 pipe 重定向（non-TTY），系統自動降級為非互動批次模式並輸出警告
Frequency of Occurrence	本地開發初次建立 suppression 清單時使用，通常每個新專案執行一次；後續違規出現時按需使用，整體低頻
Miscellaneous	是否需要提供 --accept-all 選項，一次 suppress 所有 Violation？ Suppression 是否應設有效期限，避免長期堆積技術債？

UC-04: Suppress a Violation (Brief)

Developer 執行 arch-checker suppress --id <constraint-id> --reason <text>。系統將 suppression 記錄（constraint ID、reason、timestamp）以 append 方式寫入 .arch-checker-suppress.yaml。下次執行 UC-01 時，於 Step 8 過濾生效。

UC-05: Load Style Profile (Brief — Subfunction)

UC-01 與 UC-03 的 <<include>> 子流程。系統讀取 --profile <yaml-path> 指定的 YAML 檔，透過 ProfileLoader 解析，並以 ProfileValidator 驗證規則定義。成功後將 Profile 載入記憶體，供各 constraint checker 使用。

2.6 Non-Functional Requirements and Constraints

ID	Category	Requirement
NFR-01	Performance	完整的 compliance check（UC-01）掃描 1,000 個 .java 檔案必須在 10 秒內完成（一般開發機）
NFR-02	Usability	所有錯誤訊息必須包含：錯誤原因、位置（檔案路徑 / 行號）、建議修正動作
NFR-03	Portability	工具需在 JDK 17+ 環境（Windows / macOS / Linux）上運行，不依賴特定 IDE
NFR-04	Extensibility	新增 ConstraintChecker 子類別不需修改 ArchChecker 或 StyleProfile（Open/Closed Principle）
NFR-05	CI Compatibility	exit code 遵循 POSIX 標準（0 = success，非 0 = failure），可直接整合 GitHub Actions 與 Jenkins
NFR-06	Testability	每個 ConstraintChecker 子類別必須可在不啟動完整 CLI 的情況下獨立進行 unit test

2.7 Glossary

Term	Definition
Style Profile	使用者以 YAML 格式定義的架構規則集合，描述特定專案的架構約束；工具本身不提供任何內建 Profile
ArchitectureConstraint	Style Profile 中的單條約束規則，為 NamingRule、DependencyRule、SupertypeRule、PackageRule 的抽象父類別
SupertypeRule	一種 ArchitectureConstraint，要求特定 package 內的 class 必須 implement 指定 interface 或 extend 指定 class；以 simple name 比對，不進行 fully qualified name resolution
Violation	一筆架構違規記錄，包含：檔案路徑、行號、規則 ID、說明文字
ViolationReport	單次 check 執行的完整輸出，彙整所有 Violation 與 summary
AST (Abstract Syntax Tree)	Java 原始碼的抽象語法樹表示，由 JavaParser 解析產生，供各 ConstraintChecker 進行分析
CompilationUnit	JavaParser 解析單一 .java 檔案後產生的根節點物件
Suppression	開發者明確標記「此 Violation 已知且接受」的機制，使其在後續報告中不再出現

Term	Definition
exit code	CLI 程式結束時回傳給 shell 的整數碼（0 = pass，1 = violations found，2 = error）
Package Pattern	規則定義中用來比對 Java package 名稱的萬用字元字串（如 *.ui.*）；只允許英數字、*、.

2.8 Software Environments

Item	Choice	Rationale
Development Language	Java 17 (JDK 17+)	課程規定 OO 語言；LTS 版本穩定
AST Parser	JavaParser 3.x	純 Java，API 直覺，無需 JDK tools
CLI Framework	picocli	輕量，annotation-driven，與 Java 17 相容
Profile Format	YAML (SnakeYAML)	可讀性高，易於手工編輯
Test Framework	JUnit 5	課程要求 unit testing
Build Tool	Maven	依賴管理方便

4. Measurement

Student ID	Name	HW	Date	Duration
113598009	李俊威	HW#1	115/03/03 15:00 ~ 18:00	3H
113598009	李俊威	HW#2	115/03/12 15:00 ~ 21:00	6 H